



Laboratório 05: Associação entre classes e biblioteca de terceiros

12/05/2026

1 Relógio digital

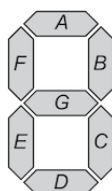
O relógio digital é uma representação gráfica do tempo, permitindo aos usuários visualizar as horas, minutos e segundos. Na Figura 1 é apresentado um exemplo de relógio digital. Esse relógio é composto por 6 *displays* de 7 segmentos, sendo que cada *display* é responsável por exibir um dígito do horário. Assim, no relógio, os dois primeiros *displays* representam as horas, os dois seguintes representam os minutos e os dois últimos representam os segundos.

Figura 1: Relógio digital exibindo 7 horas, 0 minutos e 30 segundos



Cada *display* de 7 segmentos é composto por 7 segmentos que podem ser ligados ou desligados individualmente para que possa representar um número ou letra. Por exemplo, se desejarmos representar o número 2 em um *display* de 7 segmentos, então teríamos que ligar somente os segmentos: A, B, G, E e D (veja Figura 2).

Figura 2: Display de 7 segmentos



Fonte: https://pt.wikipedia.org/wiki/Display_de_sete_segmentos

Para esse laboratório, você irá implementar classes Java para imitar o comportamento de um relógio digital, utilizando a biblioteca `algs4`¹ para representar graficamente os relógios digitais. A área de desenho (*canvas*) com o relógio deve ser atualizada periodicamente a cada 1 segundo, permitindo ao usuário acompanhar a progressão do tempo pela observação do relógio.

Você deve criar um projeto Java com gradle e implementar as classes necessárias para representar o relógio digital, garantindo que a solução respeite o princípio da divisão de responsabilidades (SOC, *Separation of Concerns*) e da responsabilidade única (SRP, *Single Responsibility Principle*). Ou seja, não deve-se criar uma única classe de forma que uma instância dessa classe assuma responsabilidades que seriam de outros objetos (lembre-se do exemplo Agenda de Contatos apresentado na aula de associação entre classes).

Mas antes de começar a implementação, você deve criar um diagrama de classes UML (perspectiva de implementação) para planejar a estrutura do seu código. Esse diagrama pode ser feito com Mermaid e ficar salvo no arquivo `Readme.md`, ou pode ser feito com uma ferramenta de modelagem UML e salvo como uma imagem.

Abaixo estão descritos os requisitos para a implementação do relógio digital. Faça uso desses requisitos para planejar a estrutura do seu código e criar o diagrama de classes UML.

¹<https://github.com/emersonmello/algs4>

1. Ao instanciar um relógio deve-se obrigatoriamente informar as coordenadas (pontos x e y em um plano cartesiano) onde o relógio deverá ser desenhado na área de desenho (*canvas*), o tamanho do relógio, a cor a ser usada para representar os segmentos ligados (use uma cor com brilho mais intenso para representar o segmento ligado e a mesma cor com menos brilho para representar um segmento desligado), a hora, o minuto e o segundo inicial do relógio.
 - Se forem fornecidos valores inválidos para hora, minuto ou segundo, então o relógio deverá ser instanciado com 0 horas, 0 minutos e 0 segundos
 - O relógio poderá ter tamanho mínimo igual a 1 e máximo igual a 3, cabe a você fazer a relação desses valores com a dimensão exata em pixels (por exemplo, 1 indica que cada segmento terá comprimento de 50 pixels na tela, já 3 indica 150 pixels). Se forem fornecidos valores inválidos para o tamanho do relógio, então o relógio deverá ser instanciado com tamanho igual a 2
2. Na classe com método `public static void main(String args[])`, por exemplo *App.java*, deve-se criar uma área de desenho (um objeto da classe *Draw*) com tamanho de 1000 pixels e ajustar a escala dos eixos para o intervalo de 0 a 1000. Esta classe será a responsável por periodicamente (a cada 1 segundo) limpar a área de desenho (*canvas*) e invocar o método do relógio digital para que esse atualize seu contador e se desenhe novamente na área de desenho.

2 Material de apoio

2.1 Importando a biblioteca algs4 em um projeto Java com gradle

1. Baixe o arquivo *algs4.jar* da biblioteca *algs4* e coloque-o na pasta *libs* do seu projeto Java com gradle. Se a pasta *libs* não existir, crie-a na raiz do seu projeto ou dentro da pasta *app*.
2. Adicione a seguinte dependência no arquivo *build.gradle.kts* do seu projeto para importar a biblioteca *algs4*:

```
dependencies {  
    implementation(files("libs/algs4.jar"))  
}
```

3. Após adicionar a dependência, sincronize o projeto para que o gradle reconheça a biblioteca *algs4*.
4. Se estiver usando git, garanta que os arquivos JAR contidos dentro do diretório *libs* estejam sendo rastreados pelo git, ou seja, que eles não estejam listados no arquivo *.gitignore*. Para isso, adicione no *.gitignore* a seguinte linha:

```
# Não ignore os arquivos JAR contidos dentro do diretório app/libs  
!**/app/libs/*.jar
```

2.2 Contador de 0 até 10 usando a biblioteca algs4

Na Listagem 1 é apresentado um exemplo sobre como executar uma rotina periódica para limpar a área de desenho (*canvas*) e desenhar o valor de um contador. Foi feito uso do método `Thread.sleep()` para fazer o aplicativo dormir por 1 segundo a cada rodada do laço que é responsável por contar de 1 até 10.

Listagem 1: Contador com *algs4*

```
import edu.princeton.cs.algs4.Draw;  
import java.util.concurrent.TimeUnit;  
  
public class App{  
    // Não iremos tratar as possíveis exceções de execução do programa, por isso o throws Exception  
    public static void main(String[] args) throws Exception{  
        Draw desenho = new Draw();  
        desenho.setCanvasSize(800, 800);  
    }  
}
```

```

desenho.setXscale(0, 800);
desenho.setYscale(0, 800);

// Toda ação de desenho acontecerá em um buffer secundário
// e esse buffer só será visto depois que for invocado o método show()
desenho.enableDoubleBuffering();

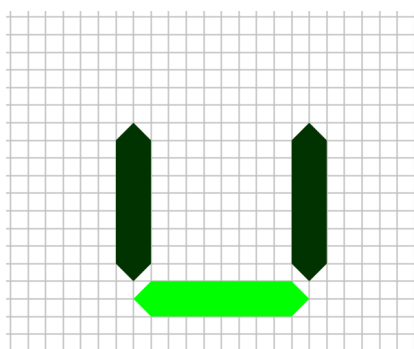
for (int i = 1; i <= 10; i++) {
    // limpando a área de desenho
    desenho.clear();
    // escrevendo o valor de i na coordenada (500,500)
    desenho.text(500, 500, "" + i);
    // Trocando o buffer para exibir o que foi escrito
    desenho.show();
    // Dormindo por 1 segundo
    TimeUnit.SECONDS.sleep(1);
}
} // fim do main
} // fim da classe

```

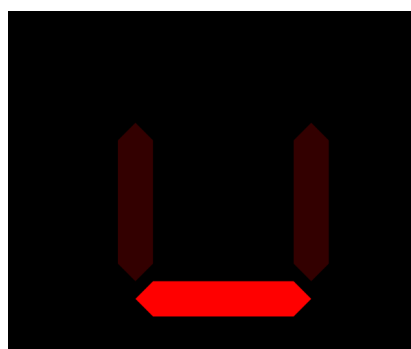
3 Desenhando polígonos preenchidos com a biblioteca algs4

Cada segmento do *display* de 7 segmentos pode ser desenhado como um polígono, assim você poderá usar o método `filledPolygon(double[] x, double[] y)` da classe `Draw`. Na Figura 3 são apresentadas capturas de tela obtidas a partir do código Java apresentado na Listagem 2.

Figura 3: Captura de telas com desenho de três polígonos



(a) Fundo branco com grade para facilitar visualização durante o desenvolvimento



(b) Fundo preto sem grade, ideal para entrega do projeto

Listagem 2: Desenhando polígonos com biblioteca algs4

```

import edu.princeton.cs.algs4.Draw;
// No VSCode é necessário habilitar o desenvolvimento com AWT
// Veja mais em https://code.visualstudio.com/docs/java/java-gui#_develop-awt-applications
import java.awt.Color;

public class Poligonos{

    public static void main(String[] args){
        Draw desenho = new Draw();
        int dimensao = 800;
        desenho.setCanvasSize(dimensao, dimensao);
        desenho.setXscale(0, dimensao);
        desenho.setYscale(0, dimensao);

        double fator = 200;
        double fatorCor = 0.2;
        Color clara = Draw.GREEN;
    }
}

```

```

Color escura = new Color((int)(clara.getRed()*fatorCor),
                        (int)(clara.getGreen()*fatorCor),
                        (int)(clara.getBlue()*fatorCor));

desenho.enableDoubleBuffering();
desenho.clear(Draw.WHITE);

// Desenhando grade quadriculada
int grade = (int) fator/10;
desenho.setPenColor(Draw.LIGHT_GRAY);
for (int i = 0; i <= dimensao; i += grade) {
    desenho.line(i, 0, i, dimensao); // linha vertical
    desenho.line(0, i, dimensao, i); // linha horizontal
}

double xInicial = 300;
double yInicial = 400;

// Montando vetores com os pontos em X e em Y para desenhar um segmento horizontal
yInicial = 180;
double[] xHorizontal = {0.1*fator+xInicial, 0.2*fator+xInicial, 1.0*fator+xInicial, 1.1*fator+xInicial, 1.0*fator+
xInicial, 0.2*fator+xInicial};
double[] yHorizontal = {0.2*fator+yInicial, 0.3*fator+yInicial, 0.3*fator+yInicial, 0.2*fator+yInicial, 0.1*fator+
yInicial, 0.1*fator+yInicial};

// Desenhando o segmento horizontal
desenho.setPenColor(clara);
desenho.filledPolygon(xHorizontal, yHorizontal);

// Montando vetores com os pontos em X e em Y para desenhar um segmento vertical
yInicial = 200;
double[] xVertical = {0.1*fator+xInicial, 0.2*fator+xInicial, 0.2*fator+xInicial, 0.1*fator+xInicial, 0*fator+
xInicial, 0*fator+xInicial};
double[] yVertical = {0.2*fator+yInicial, 0.3*fator+yInicial, 1.0*fator+yInicial, 1.1*fator+yInicial, 1.0*fator+
yInicial, 0.3*fator+yInicial};

// Desenhando o segmento vertical
desenho.setPenColor(escura);
desenho.filledPolygon(xVertical, yVertical);

// Desenhando outro segmento vertical, porém com x deslocado em 'fator' pixels
for (int i = 0; i < xVertical.length; i++) {
    xVertical[i]+=fator;
}
desenho.filledPolygon(xVertical, yVertical);

// Trocando o buffer para exibir o que foi desenhado
desenho.show();
} // fim do main
} // fim da classe

```